

~~cat arboretum.txt && cat | ls | grep "hrant"~~

ls | grep "hrant" | tee arboretum.txt

```

int main () {
    int fd[2];
    if (pipe(fd) == -1) {
        perror("pipe error");
        exit(EXIT_FAILURE);
    }
    if (fork() == 0) {
        dup2(fd[1], 1);
        close(fd[1]);
        close(fd[0]);
        execlp("ls", "ls", NULL);

        perror("ls_error");
        exit(EXIT_FAILURE);
    }
    // else wait pid (-1, &status, 0);
    int fd2[2];
    if (pipe(fd2) == -1) {
        perror("pipe error");
        exit(EXIT_FAILURE);
    }
    if (fork() == 0) {
        dup2(fd2[1], 0);
        close(fd2[1]);
        close(fd2[0]);
        dup2(fd[1], 1);
        close(fd[1]);
        close(fd[0]);
        close(fd2[0]);
        execlp("grep", "grep", "hrant", NULL);

        perror("grep_error");
        exit(EXIT_FAILURE);
    }
    // else wait pid (-1, &status, 0);
    close(fd[0]);
}

```

```
int fd1 = open("arboresum.txt", O_CREAT | O_WRONLY, O_TRUNC, 2,  
               | O_TRUNC, 0644);
```

```
if (fork() == 0) {  
    dup2(fd1, 1);  
    close(fd1);  
    dup2(fd2[0], 0);  
    close(fd2[0]);  
    close(fd2[1]);  
    execvp("tee", "tee", NULL);  
    perror("tee_error");  
    exit(EXIT_FAILURE);  
}
```

```
} else waitpid(-1, &status, 0);
```

```
waitpid(-1, &status, 0);  
waitpid(-1, &status, 0);  
waitpid(-1, &status, 0);  
close(fd1);  
close(fd2[1]);  
close(fd2[0]);  
return 0;
```

```
}
```

1.a) Obrazložiti in primerjati ASCII in UTF-8 kodiranje znakov. Koliko bajtov zasede niz »puščava«.

ASCII - 7-bit encoding

33 kontrolnih znakov

128 - 33 znakov = 95 znakov

$2^7, i \in [0, 2^7 - 1]$

verjetno niz brez character
UTF-8 - 7-bit encoding

»puščava« ni mogoče napisati v ASCII, saj č, š, ě, đ nimajo vrednosti.

»puscava« je mogoče zapisati v ASCII, dolžine 7 znakov, 7 byte.

»puščava« je mogoče zapisati v UTF-8, kjer določeni znaki (character) so manjše velikosti kot drugi, ASCII v UTF-8 so 1-byte, š, ě pa 2-byte.

»puščava« - UTF-8

$1 + 1 + 2 + 2 + 1 + 1 + 1 = 9$ byte.

'p', 'u', 'š', 'č', 'a', 'v', 'a'.

UTF-8, vključuje šumnike

»puščava«, $1 + 1 + 1 + 1 + 1 + 1 + 1 = 7$ - byte.

b) Obrazložiti naslovni prostor večnitnega procesa v (programi v C). Nariši diagram prostora.

d) Na sliki je podana razdelitev diska /dev/sda (vse velikosti so v GiB). Največ kako veliko particijo lahko še dodatno naredimo?

sda1: 20 prsto: 20 sda4: 10 prsto: 40 sda3: 10 prsto: 10

$\max(20, 40, 10) = 40$ GiB. Naredim lahko največ particijo velikosti 40 GiB.

Particija (določena v tabeli particij) ne more prebožiti druge particije).

skozi čez particijo sda4. - Prepovedano.

prsto: 20 sda4: 10 prsto: 40

sda2: 20 sda4: 10 sda2: 40 - Prepovedano.

Zaporedje prstih particij lahko določi razširjeno particijo (extended partition).

prsto: 10 prsto: 30 prsto: 20

sda2: 10 / sda2: 30 sda2: 20 - Pravilno.

EBR0E... → EBR1E...

sda2: 60 / Napako.

Zaporedje, določeno kot razširjena particija (extended partition), je zaporedje EBR (Extended - Boot - Record), ki je logična particija (manjša tabela - array -) s kazalcem (pointer) do naslednje EBR.

```
exit(0);  
}
```

5.

a) ...

```
cat -n dnevnik.txt | grep "zaklad"
```

b) /etc/shadow

Drugi stolpce encrypted password

```
cd /etc/shadow | cut -d ':' -f2 /etc/shadow
```

root user. Non-root user ne more dostopati do
hashtanik (hashed) gesel, v primeru dostopa
bošje dostopati do hashtanik gesel (user password).

c)

Najde pid xmap

```
ls -la /proc/#(pgrep xmap)/fd/
```

Does not need sudo

file descriptors

Imenovana cev

Do rje dostopa vsek proces, ki pozna path. cevi, sistemski klic `mkfifo(path, mode)`. V primeru, da mode označi, da drugi proces morda nima permission, vrne EACCESS (Error Access).
mode, običajno 0666.

mode == 0666

Marked.

Special bits: Owner, Group (member of the file's group), Others.

4 -- setuid - execute with owner-privilege

2 -- setgid -

1 -- sticky-bit

Permissions - digit(s).

4 -- read

2 -- write

1 -- execute

rw-r--r-- (octal 0644)

Owner Group others

Imenovana cev (FIFO pipe)

`mkfifo(path, mode)`, `open()`, `read()`, `write()`, `clone()`, `unlink()`.

Anonimna cev (Anonymous pipe)

`pipe(fd)`, `read()`, `write()`, `clone()`.

File-descriptor.

Če se `unlink()` uporabi za odstranitev cevi, ontare cev delujoča med procesi, ki imajo odprt kanal (channel) = `open()`. Po `unlink()` noben proces ne more odpreti kanala (channel).

Če več hi `fd[i]`, `clone(fd[i] - value - 1)`, se cev (pipe) odstrani.

`read()` in `write()` v anonimni cevi (anonymous pipe)

in v imenovani cevi (named - FIFO - pipe) lahko blokirata.

Če `read()` bere iz praznega medpomnilnika (se vedno vrne 0), če prebrane, vrne EOF (tudi če `read()` prebere zadnjo +1 vrednost), saj je +1 po `End-Of-File`.

b) Naj bo velikost grozda datotečnega sistema FAT enaka 4 KiB. Koliko grozdov zaseda datoteka velikosti 10 000 B?

$$4 \text{ KiB} = 4 \cdot 1024 \text{ B} = 4096 \text{ B}$$

$$\lceil 10.000 \text{ B} / 4096 \text{ B} \rceil = \lceil 2,441 \rceil = 3 \text{ grozda (cluster)}$$

Vrsta datotečnega sistema
FAT, ext 4, ...

Grozid - cluster - je logična enota (saj je memory mapped) hardware spomina (v disku - disc space).

Spremenil naloge, temveč
postopek reševanja ostane enak.

Stran - page - je logična enota RAM. (Random-Access-Memory).

c) Nariši primer tabele FAT, ki hrani datoteko rezine.jpg, v grozdih = indeksih 9, 7, 2, 6 in 8, zaporedoma.

Index	Value	Glej tudi vrednosti. Datoteka je zapisana v več grozdih. FAT hrani naslednje.	
1	→ 6		
2	→ 8		
3	→ 7		
4	→ 2		
5	→ EOF		
6	→ 9		
7	→ 7		

Prav, vedno preveri.

b) cat diera.txt | cd | cat -n Neppravilno.

xargs -a diera.txt | wc -l
 Read from file instead of standard input
 count
 newLines
Build, and execute commands from standard input.

c) awk '{ if (\$1 > max) { print max } }' veraica.txt

Napácho.

awk '{ \$1 > max } ; END { print max }' veraica.txt

Napácho.

awk '{ if (\$1 > max) max = \$1 } ; END { print max }' veraica.txt

d) awk '{ print \$2 }' /etc/shadow

3.

$x = 100$

10 hti $x = x + 1$, 10 hti $x = x - 1$

Njena pričakovana vrednost je 100, saj 10 hti inkrementira x , in 10 hti dekrementira x . Vendar, zaradi prepletanja procesov, ki

delujejo sočasno, določen proces lahko prebne vrednosti, preden je drugi proces inkrementiral oz. dekrementiral, zato je dejanska vrednost manj nedriveljav, vendar znotraj intervala $[100 - 10, 100 + 10] = [90, 110]$

V večprocesnih sistemih ne velja z izjemo jedra (kernel), zaradi virtualizacije protokola procesa RAM
velja v večprocesnih programih. (vsak program ima določen interval

Ni potrebno znati
Kdaj ni čaka.

RAM, ki ohlajajo zunanje
spremembe sprememb (izjemo kernel
ali namerno) spremembo v kodu.

vedile take conditions (in intervalu

Kot procesa n0 na RAM procesa m opravljajo ukaze z izmenjevanjem
ključa mutex, da bi imeli pravilno delo procesorskega časa
če določena nit prebere vrednost - preden jo je druga nit zapisala,
prebere predhodno vrednost, zato je rezultat na intervalu $[100-10, 100+10]$
 $[90, 110]$.

Brez sinhronizacije mutex,

c) Procesi in niti brez sinhronizacije ne morejo imeti tregana
stanja, saj morajo dojem (deadlock), in živi (objem?) (livelock)
temeljita na nepravilni sinhronizaciji, pri mutnem dogemu (deadlock)
oba procesa čakata na ključ, in pri žnem objemu (livelock) pa
zaporedno, ob istem trenutku poskušata dobiti ključ (in zavržeta
konflikta, zato počakata, in poskušata ponovno dobiti (prejeti) ključ.

Procesi lahko imajo tregana stanja le če si delijo
spremenljivke (globalne spremenljivke, heap se dedujejo v obliki
kopij).

d) vrsta = new semaphore(1).

Vrsta nit / proces
wait(vrsta)

$x \geq x + 1$

signal(vrsta).

2.
dha. 8

a) ASCII: 7 bit encoding, 128 characters, 32 control codes, and 128-32 characters (non-control-code characters).

UTF-8

1-byte for ASCII

1-4 byte

2-byte for other characters

4-byte symbol, kot je smile-symbol

Učinkovito za zahodne jezike, vendar zahtevna implementacija.

UTF-16

3-byte znaki v UTF-8, 2-byte znaki v

UTF-16

2-4 byte znaki.

Prilagojeno vzhodnim jezikom.

UTF-32

Vedno 4-byte, pronačljiva neučinkovitost, (vendar) preprosta implementacija.

b) Ukazna lupina je vmesnik (interface), ki omogoča komunikacijo med uporabniškim programom in jedrom operacijskega sistema (kernel).

Notranji ukazi so ukazi, ki so vgrajeni v ukazno lupino, medtem ko so zunanji ukazi, ukazi naloženega programa, ki ga lupina lahko izvede.

Win.

c) Windows

Linux

.exe

.so

.text

.text

.docx

.doc

.msi

.apk, .rpm, ...

Pisni izpit 13. junij 2012

1.

Razvrščevalnik iz množice procesov Ready Queue izbere proces, ki bo naslednji dobil procesorski čas, in je (najpogostejši) algoritem RR (Round-Robin), Priority scheduling,

(Job Scheduler)

Deli se na dolgoročni razvrščevalnik, ki določa, kateri program se bo naložil v pomnilnik in postal proces, in kratkoročni razvrščevalnik (CPU Scheduler).

Dodeljevalnik (Dispatcher) je nizkonivojska (low-level) koda (code), ki izvede prelop konteksta (context switch), ki shrani stanje starega procesa, in nalaganja stanja naslednjega procesa, ki je izbral razvrščevalnik (Job Scheduler).

Ni potrebno poznati podrobnosti (vedeti). Morda ni točno.

2.

SJF (Shortest-Job-First)

✓ celoti opravi proces, ki ima najkrajše trajanje, nato izbere naslednji proces (dodaten ni več procesov za opraviti).

Operacijski sistem; Procesi

1

Če starš završi, in otrok naredi `exit(...)`, pokliče v `SIGCHLD`, in postane zombie. V primeru, da se starš zbudi, in sledi `waitpid()`, jedro operacijskega sistema (kernel) shrani `SIGCHLD` signal (shrani signal zaradi `SIGCHLD`, ki sledi) vrne `waitpid(...)`, zato starš prevzame (collect) proces.

Če starš konča, naredi `exit()`, kernel naredi `waitpid(...)` za starša (zombie proces), pred otrokom, otrok postane sirota (orphan), in je otrok init. Ko otrok konča, ga init prevzame (collect), ko otrok naredi `exit()`.

jobs
CTRL + Z
fg <jid>
bg <jid>

Posel (job) je opravilo, ki je v celoti pripravljeno za izvajanje.

`SIGHUP` (Signal Hang Up), ko

Kernel pošlje `SIGHUP` otrokom starševskega procesa, ki je terminal, kar prepozna, saj ga upravlja (manage), saj je terminal otroški proces kernel.

Bash je del terminala, kjer je terminal le I/O za bash. Oba bash in kernel lahko prejmeta `SIGHUP`. Bash dokonča vse svoje procese, ki so premaknjeni na kernel.

`open()` vrne št. datotečnega descriptorja.

`read()`, `write()` vrne št. obdelanih bajtov.

oz. št. elementov character
v popolnjeni tabeli (array).

time ls -l /etc > ls-out.txt

real time večji od sistema user time (posel se opravlja v uporabniškem načinu), in sys time (čas, ki se posel čaka na signal iz jedra operacijskega sistema - kernel-).

tr ... reads input, and replaces input character with output character

echo 1 2 3 | tr 1 5

↓
Replace all '1' with '5'.

6. a) ... Napiši ukaz, ki izpiše predzadnjo uspešno prijavo v sistemu.

```
last | head -n -2 | tail -n -1.
```

b) Napiši ukaz, ki izpiše vse trenutno prijavljene uporabnike, razen tvojega.

, Remove from output.

```
lastb -n 20 | grep -v "$(whoami | cut -c1-7)"
```

last, lastb .log file
(različna .log file)
vsebuje \$(whoami),
prvih 7 character, in '*'.

cut username string +
\$(whoami) to first 7
characters.

Čb \$(whoami) = Lsarjanovic,
je v last, lastb log file
"Lsarjan*"

8 chars, oz. 7 chars, in '*'.

c) Poskusi, da storilec ni več prijavljen v sistemu, da ne more več delati škode. ^{izpiše} Napiši ukaz, ki izpiše vse trenutno prijavljene uporabnike, razen tvojega.

`users | grep -v "$(whoami)"`

Users folder ("/usr/bin/users") ima string "\$(whoami)" zapisan v celotni.

Da najdem folder-path, zapisem

which users, oz.

command -v users

d) ...oscoin.log v domačem imeniku - preveri čas zadnje spremembe datoteke. Napiši ukaz, ki izpiše ta čas.

`date -r ~/oscoin.log`

Domači imenik (home directory).

`ls -l ~/oscoin.log`

e) Napiši ukaz, ki primerja vsebino obeh datotek.

`cmp oscoin.log /media/backup/oscoin.bkp`

Ukaz

Morda sta sokrat in jama jama@ sokrat, vendar
maj bi bilo, kot je zapisano.

sokrat@jama:~>

sokrat -- user-name

'@' -- (standard) separator between user-name and host-name

jama -- hostname (device-name)

!:" -- separator between ^{what} who, and ^(where to) where

'/' -- home-directory

'>' -- prompt-symbol pldro before user-type-input.

1.

d) awk '{ sum += \$1 } END { print sum }' "#keyfile"

1.a) Naloga mikro-jedra (je, da upravlja in nadzoruje mikro gonilnike.

mikro jedro (microkernel - v Linux-),

mikro gonilnik (driver - v Linux+).

Ne, mikro jedro (microkernel) le omogoča komunikacije, IPC, med mikro gonilniki, in jih ne nadzoruje.

Mikro gonilnik (driver - v Linux+)

je samostojna logična enota, koda (code), software za določen hardware (omogoča komunikacijo s hardware).

b) Prikazni strežnik je del grafične lupine, ki skrbi za komunikacijo z uporabniškimi programi glede dogodkov povezanih z uporabniškim vmesnikom.

Da, saj grafična lupina (graphical shell) sestavljajo prikazni strežnik (display server), ki je mehanizem, ki ureja input, output, in focus (skrbi za komunikacijo z uporabniškimi programi), in window manager (npr. kwin), desktop environment, (npr. KDE), in widget toolkit, ki predstavljajo prikaz podatkov.

Ni potrebno poznati podrobno (vedeti), morda ni tačno.

c) Sistemsko virtualizacijo delimo na virtualizacijo tipa 1, ki se izvaja neposredno na strojni opremi in virtualizacijo tipa 2, ki temelji na gostiteljskem OS.

Da.

d) Gočnost brez vzporednosti pomeni, da lahko pri izvajanju strojne kode pride do prepletanja ukazov, do njihovega prekrivanja pa ne.

Da, saj je prekrivanje mogoče le, če se izvajajo ukazi vzporedno (več kot en procesor).

e) Pri raziskovalniku »Linux $O(1)$ «, aktivna tabela vsebuje procese, ki se trenutno dejansko izvajajo na procesorju, pretečena tabela pa procese, ki so že porabili svojo časovno težino.

Da.

2) "

a) Kaj je sistemski vbric? Kaj je njegova ovojna funkcija?

Sistemski vbric je mehanizem, preko katerega uporabniški program zahteva jedrno storitev.

Ovojna funkcija (wrapper function) pripravi argumente, izvede sistemski vbric, v jedru so argumenti preverjeni in vrne vrednosti.

b) Koliko bitov zahteva standardna britanska ASCII? Kako so v njej kodirani slovenski črnički?

ASCII zahteva 7-bitov. Slovenski črnički niso kodirani v ASCII, saj bi se vrednosti slovenskih črničkov zasedajo drugi simboli, vendar v UTF-8 vključuje slovenske črničke.

c) Kako v tekstovnih datotekah kodiramo znak za novo vrstico?
Primerjaj pristopa v Windows in Linux OS.

'\r' v Windows.

Pitag.

'\n' v Linux.

d) Objasni, kaj je ukazna lupina? Kateri je osnovni vhodni element ukazne lupine

(shell)
Ukazna lupina je vmesnik (interface) med uporabnikom in jedrom operacijskega sistema.

Ukazna vrstica je osnovni vhodni element ukazne lupine.

Command line (string, oz. character array).

e) Kaj se zgodi, ko v lupini zahtevamo izvedbo zunanjega ukaza, npr. zaženemo ukaz ls? Objasni in pojasni s tem povezane sistemske blize.

Lupina stvari opravi fork(). Če je fork() uspešen izvede execve("program-path"); ^{z novim brenubrega} procesa, oboka je ^{z novim programom} prepiran, medtem ko stari vrtano z waitpid(), ~~execve("program-path")~~, ki ga najde s ~~\$PATH variable~~ da konča proces, ko naredi ~~exit(0)~~, oz. ~~exit(1)~~, če ~~execve()~~ ni našel programa.

fork(value)

gol je naključno generiran niz znakov, ki ga dodamo geslu, pred zgoščevanjem, da bi težili iskanje gesla z rainbow tables (enaka gesla zaradi različnih soli imata različni hash vrednosti). V Linux je v /etc/shadow (se uporabljajo le hvani uporabniška gesla)

3.) a) Opišite in primerjajte ločevanje med datotekami različnih pomnilnih naprav v Windows in Linux OS.

Windows ima tabelo, oz. polje array Object Manager, kjer hrani drevesa posameznih pomnilnih naprav (metadata), in polje pointerjev, medtem ko Linux ne doda na pristo mesto v rpo folder, '/' kazalec (pointer).

Zgradba je enaka, kar je kazalec (pointer) do metapodatkov (dreves), le prikaz je drugačen. Ni potrebno poznati podrobnosti (vedeti). Morda ni točno.

b) Opišite princip COW (kopiraj do pisanja, copy-on-write), kje in zakaj se uporablja v Linux OS?

$$T_3 = (1 - \alpha)$$

$$\alpha = 0,5$$

i	1	2	3	4
t_i	11	4	2	20
T_i	18	6	4	12

$$\begin{aligned} T_1 &= \alpha \cdot t_1 + (1 - \alpha) \cdot T_0 \\ &= 0,5 \cdot 11 + (1 - 0,5) \cdot 5 \\ &= 5,5 + 2,5 \\ &= 8,5 \end{aligned}$$

$$\begin{aligned} T_3 &= \alpha \cdot t_3 + (1 - \alpha) \cdot T_2 \\ &= 0,5 \cdot 2 + (1 - 0,5) \cdot 6 \\ &= 1 + 3 \\ &= 4 \end{aligned}$$

$$\begin{aligned} T_2 &= \alpha \cdot t_2 + (1 - \alpha) \cdot T_1 \\ &= 0,5 \cdot 4 + (1 - 0,5) \cdot 8,5 \\ &= 2 + 4,25 \\ &= 6,25 \end{aligned}$$

$$\begin{aligned} T_4 &= \alpha \cdot t_4 + (1 - \alpha) \cdot T_3 \\ &= 0,5 \cdot 20 + (1 - 0,5) \cdot 4 \\ &= 10 + 2 \\ &= 12 \end{aligned}$$

1.13 $\alpha = 0,2$ $T_0 = 10$

i	1	2	3	4
t_i	10	15	16	22
T_i	10	11	12	14

$$T_1 = \alpha \cdot t_1 + (1 - \alpha) \cdot T_0$$

$$\begin{aligned} T_1 &= 0,2 \cdot 10 + (1 - 0,2) \cdot 10 \\ &= 2 + 8 \\ &= 10 \end{aligned}$$

$$\begin{aligned} T_3 &= \alpha \cdot t_3 + (1 - \alpha) \cdot T_2 \\ &= 0,2 \cdot 16 + (1 - 0,2) \cdot 11 \\ &= \frac{1}{5} \cdot \frac{16}{1} + 0,8 \cdot \frac{11}{1} \\ &= \frac{16}{5} + \frac{8,8}{1} \\ &= 3\frac{1}{5} + 8\frac{4}{5} \\ &= 3,2 + 8,8 \\ &= 12 \end{aligned}$$

$$\begin{aligned} T_2 &= \alpha \cdot t_2 + (1 - \alpha) \cdot T_1 \\ &= 0,2 \cdot 15 + (1 - 0,2) \cdot 10 \\ &= 3 + 8 \\ &= 11 \end{aligned}$$

$$\begin{aligned} T_4 &= \alpha \cdot t_4 + (1 - \alpha) \cdot T_3 \\ &= 0,2 \cdot 22 + (1 - 0,2) \cdot 12 \\ &= \frac{1}{5} \cdot \frac{22}{1} + 0,8 \cdot \frac{12}{1} \\ &= \frac{22}{5} + \frac{48}{5} \\ &= \frac{22}{5} + \frac{48}{5} = \frac{70}{5} = 14 \end{aligned}$$